AI-Powered Web Security Learning Assistant

Project Handbook • Research, Secure Coding, Quiz and Ethics

Audience: Second-year CSE students learning web development

Research summary

OWASP Top 10:2025 is an awareness document, not a complete audit. Its categories are Broken Access Control; Security Misconfiguration; Software Supply Chain Failures; Cryptographic Failures; Injection; Insecure Design; Authentication Failures; Software or Data Integrity Failures; Security Logging & Alerting Failures; and Mishandling of Exceptional Conditions.

NIST SSDF SP 800-218 v1.1 recommends integrating secure development practices into the lifecycle. MDN guidance includes HTTPS, CSP, secure cookies, input validation, safe output handling, strong authentication, and dependency control. Flask documents additional controls including CSRF protections, security headers, and resource limits.

AI is useful for explaining code and suggesting defensive fixes, but may hallucinate or use outdated advice. Verify every claim against primary sources, test changes locally, and do not share secrets with AI.

Secure-coding checklist

■ Validate all inputs on the server using allowlists, type checks, and sensible length/range limits.

■ Use parameterized database queries or a safe ORM; never concatenate untrusted input into SQL.

■ Use safe output APIs (e.g., `textContent`) and context-aware output encoding; sanitize only when rich HTML is required.

■ Treat file uploads, URLs, headers, cookies, JSON, and hidden form fields as untrusted.

■ Store passwords with a maintained password-hashing function (Argon2id, scrypt, bcrypt, or framework-supported equivalent).

■ Use secure session handling; rotate sessions after login and invalidate them on logout where applicable.

■ Check authorization server-side for every protected action and object, not just at the UI.

■ Use MFA/passkeys where appropriate and rate-limit authentication attempts.

■ Use HTTPS in deployment; never disable TLS certificate verification to “fix” a connection.

■ Keep secrets out of source control; use environment variables or a secret manager.

■ Disable debug mode and verbose stack traces in production.

■ Set suitable cookie flags (`Secure`, `HttpOnly`, `SameSite`) and review CSRF protections for state-changing requests.

■ Apply least privilege to database accounts, services, and file permissions.

■ Pin/review dependencies and remove unused packages.

■ Check advisories and update dependencies using a controlled process.

■ Protect CI/CD secrets and restrict workflow permissions.

■ Verify downloaded artifacts and use integrity controls where appropriate.

■ Return generic client-facing errors; keep sensitive diagnostics in restricted logs.

■ Log security-relevant events without logging passwords, tokens, or unnecessary personal data.

■ Set request/body/time/resource limits and handle exceptional conditions safely.

■ Add tests for both expected and rejected inputs and for authorization boundaries.

■ Ask AI for defensive explanations and tests, not exploits or attack tooling.

■ Verify each claim against official documentation.

■ Review AI code manually, run tests, and check for outdated APIs.

■ Never paste secrets, private data, or code you are not permitted to share.

■ Record hallucinations, unsafe suggestions, and corrections.

Defensive guide

Audience: Second-year CSE students learning web development

Web applications accept data, identify users, access databases, and expose functions over networks. A small coding mistake can expose private information or let a user perform an action they are not allowed to perform. OWASP Top 10:2025 is an awareness document describing ten important categories of web application risk. It is a starting point, not a complete list of every possible weakness and not a substitute for threat modeling or testing.

SQL injection can happen when an application joins untrusted input directly into a database query. The database may interpret part of the input as query syntax instead of data. A safer pattern is to use parameterized queries or a well-configured ORM. The database driver binds values separately from the query structure. Input validation remains useful, but it is not a replacement for parameterization. Also use a database account with only the permissions the application needs, and test query behavior with ordinary and invalid inputs in a local test database.

XSS is a class of issue where untrusted content is interpreted as executable browser content in a page. The right defense depends on the output context. For plain text, APIs such as JavaScript `textContent` avoid treating the value as HTML. Template engines often escape text by default, but developers must understand when escaping is disabled. If a product genuinely supports rich HTML, use a maintained sanitizer with an explicit policy. A Content Security Policy can add defense in depth, but it does not replace safe rendering. Test using harmless sample strings in your own local app; do not build or distribute exploit payloads.

Validation checks whether an input matches expected type, format, and business rules. For example, an age field might need to be a whole number in an allowed range. Prefer allowlists: define acceptable values and reject unexpected ones. Validate on the server because client-side checks can be bypassed. Validate early and give helpful error messages without exposing internal details. Validation alone cannot safely neutralize every dangerous input in every context; pair it with the correct SQL, HTML, file-upload, or command-specific defense.

Passwords should never be stored in plain text or protected with a fast general-purpose hash such as SHA-256 alone. Use a maintained password-hashing implementation designed to slow guessing, such as Argon2id, scrypt, or bcrypt, with appropriate parameters. Do not invent your own password storage scheme. Authentication verifies identity; authorization decides what that identity may access. Use secure session management, protect cookies, rate-limit login attempts, and consider multi-factor authentication or passkeys. Avoid leaking whether a particular account exists through overly specific login errors.

An application may correctly log a user in but still fail to check whether they are allowed to read or change a particular record. Never trust an ID supplied by a URL or form as proof of permission. Check authorization on the server for every sensitive action and every object, including API routes. A common safe design fetches the requested object, checks the current user's role or ownership, and only then returns or changes it. Test with two test accounts and confirm that one account cannot access the other's private sample data.

Default passwords, exposed debug pages, overly broad CORS rules, verbose errors, and missing security headers can increase risk. Use secure defaults, keep development and production configuration separate, disable debug mode in deployment, and reveal only generic errors to users. Use HTTPS and configure cookie attributes such as `Secure`, `HttpOnly`, and `SameSite` appropriately. Security headers like Content Security Policy should be tailored and tested; copying a header blindly can break an application or give a false sense of security.

Web apps rely on frameworks, libraries, packages, build systems, and CI workflows. A vulnerable or untrusted dependency can affect the whole application. Keep a clear dependency inventory, review advisories, update deliberately, remove unused packages, and protect repository and CI secrets. Lock or pin dependencies in a reproducible way when suitable for the ecosystem. Do not assume a package is safe just because it is popular, and do not install code from untrusted sources without review.

Sensitive information must be protected in transit and, where appropriate, at rest. Use HTTPS with valid certificate verification, approved cryptographic libraries, and suitable key management. Never hard-code real secrets in source code or commit them to a public repository. Do not design your own encryption algorithm. Use password hashing for passwords; encryption and hashing solve different problems. Review how keys are created, stored, rotated, and accessed.

Applications should handle errors predictably. A detailed stack trace may help a developer locally but can reveal paths, queries, versions, or secrets to a user. Return generic client errors and record necessary diagnostic

information in access-controlled logs. Logs should not contain passwords, session tokens, or unnecessary personal data. Log relevant security events, protect log access, and define how someone will respond to alerts. Set sensible size and time limits so unusual requests do not exhaust resources.

AI can explain code, suggest safer patterns, draft tests, and help beginners compare alternatives. However, AI may invent APIs, recommend deprecated practices, miss authorization logic, or confidently misstate a security claim. Ask for sources and defensive fixes, then verify claims against official OWASP, NIST, MDN, and framework documentation. Compile and test suggested code in a local environment; use peer or instructor review for important changes. Do not paste credentials, private data, or code you are not allowed to share. The assistant in this project uses a small heuristic checker, so its output is only a learning signal: it can produce false positives and false negatives.

Practice only on code you own, an isolated local lab, or authorized educational applications such as OWASP Juice Shop or DVWA. This project does not include working exploits, payload generation, remote scanning, or attack automation. The purpose is to understand risks and prevent them.

Safer code comes from layers: safe framework defaults, validation, parameterized queries, contextual output handling, authorization checks, secure configuration, dependency management, testing, and careful operations. AI is a learning aid, not an authority. Verify every important suggestion and make security part of the development process from the beginning.

Quiz and answer key

Choose one answer for each question.

1. What is the preferred defense against SQL injection when values are used in SQL?

A. Hide the query from the browser B. Parameterized queries C. Rename the table D. Use a longer URL

2. Which JavaScript API is generally safer for displaying untrusted plain text?

A. `innerHTML` B. `eval` C. `textContent` D. `document.write`

3. Why must server-side validation exist even if a form has browser validation?

A. Browser validation can be bypassed B. It makes CSS work C. It encrypts data D. It replaces authorization

4. Which is a suitable approach for password storage?

A. Plain text B. Base64 C. MD5 D. A maintained password-hashing function such as Argon2id/bcrypt/scrypt

5. Authentication answers which question?

A. What colour is the page? B. Who is the user? C. Which CSS file loads? D. How big is the database?

6. Authorization checks:

A. What the authenticated user is allowed to do B. Whether JavaScript is enabled C. The user's screen size D. Whether a package is popular

7. What should developers do with API keys?

A. Commit them to GitHub B. Put them in HTML C. Keep them in secret management/environment configuration D. Print them in logs

8. What is the best response to an AI claim that cites a security API you cannot find?

A. Trust it because it sounds confident B. Verify with official documentation and test safely C. Deploy immediately D. Delete all tests

9. What does a clean result from this project's heuristic scanner mean?

A. The app is guaranteed secure B. No configured patterns matched; manual review is still needed C. It has no bugs D. It is safe to deploy

10. Where should students practise security checks?

A. Any public website B. A stranger's server C. Their own local lab or explicitly authorized training environment D. Random IP addresses

1. B — Parameterization separates data from SQL syntax.

2. C — `textContent` inserts text rather than parsing it as HTML.

3. A — Attackers can send requests without using the page's form controls.

4. D — Password-specific hashing is designed to make guessing more costly.

5. B — Authentication verifies identity.

6. A — Authorization determines allowed actions and resources.

7. C — Keep secrets out of source control and client-side code.

8. B — Confirm claims using primary documentation and controlled tests.

9. B — Heuristic scanners are incomplete and can miss vulnerabilities.

10. C — Only test systems you own or have explicit permission to test.

OWASP Top 10:2025 Infographic

